

CHAPITRE 2 : ELEMENTS DE BASE

Les exemples et les exercices ont été testés avec l'environnement de développement **Code ::Blocks**.

I. Introduction :

La mémoire d'un ordinateur ne peut contenir que des valeurs binaires : les programmes sont donc codés en binaire pour s'appliquer à des données elles aussi codées en binaire.

Les programmes ne sont pas écrits directement en binaire, mais dans un langage de haut niveau (C/C++, Java, C#, etc) en suivant des règles d'écriture très précises (nommées règles syntaxiques). Ceux-ci sont ensuite transformés en programmes binaires exécutables par des programmes spéciaux appelés compilateurs.

Dans ce module on étudiera la programmation avec le langage C++ qui est un langage évolué et structuré. C'est en ce sens une évolution du langage C. Il possède en outre les fonctionnalités de la programmation orientée objet.

Le langage C++ possède assez peu d'instructions, il fait par contre appel à nombreuses bibliothèques.

exemples:

iostream : bibliothèque d'entrées/sorties standard

math.h : bibliothèque de fonctions usuelles des mathématiques

Le langage C++ est un langage compilé, ce qui signifie que pour arriver à un programme exécutable il faut passer par un certain nombre d'étapes dans le développement :

1-Edition du programme source : se fait à l'aide d'un éditeur de textes. Le nom du fichier contient l'extension .cpp, exemple: tp1.cpp . (Text editor)

2-Compilation du programme source, c'est à dire création des codes machine destinés au microprocesseur utilisé. Le compilateur indique les erreurs de syntaxe mais ignore les fonctions

des bibliothèques appelées par le programme. Le compilateur génère un fichier binaire, non éditable en mode « texte », appelé fichier objet: tp1.o (ou tp1.obj). (Compiler)

3-Editions de liens: Le code machine des fonctions des bibliothèques est chargé, création d'un fichier binaire, non éditable en mode texte, appelé fichier exécutable: tp1.exe. Il est réalisé par un éditeur de liens (linker)

4- Exécution du programme : Le lancement du programme. Il est réalisé par débogueur (debugger)

Ces différents outils sont regroupés au sein d'un même logiciel appelé EDI (environnement de développement intégré). On trouve des EDIs comme :

- Code::Blocks
- Dev C++
- Microsoft Visual Studio C++
- Borland C++ Builder
- ...etc

II. Structure d'un programme :

La structure d'un programme avec C++ a la forme de l'exemple suivant :

Exemple : Editer, compiler et exécuter le programme suivant:

```
// les commentaires s'écrivent derrière 2 barres obliques

#include<iostream>// Bibliothèque standard des entrées et sorties

int main() //La fonction principale du programme
{
    std::cout<< "Programmation avec C++ "; //affichage sur l'écran

    return 0; //La fonction main return un entier
}
```

La directive **#include** permet d'inclure un fichier dans un programme. **#include<iostream>** inclut la bibliothèque **iostream** qui contient les « fonctions » des entrées/sorties utilisées pour la saisie au clavier et l'affichage à l'écran.

Tout programme C++ doit comporter la **fonction principale main** qui commence par la ligne **int main()**, puis comprend le code du programme entre accolades.

```
int main()
{
    ... ;
    ... ;
    return 0 ;
}
```

La dernière ligne est du programme est l'instruction **return 0**; Elle indique seulement que la fonction **main** s'est terminée correctement sans erreur particulière.

III. L'affichage à l'écran : cout <<

L'instruction **std ::cout << "Programmation C++" ;** de l'exemple précédent permet d'afficher le message **Programmation C++** à l'écran.

Le flux de sortie standard **cout** (Console Output : sortie console), définit dans la bibliothèque **iostream**, permet d'afficher les messages à l'écran en utilisant l'opérateur <<.

Exemple :

```
#include <iostream>
int main()
{
    std::cout << "Programmation avec C++ ";
    std::cout << "Filiere SMP - S4 ";//affichage sur la même ligne
    return 0;
}
```

On peut récrire l'exemple précédent de façons plus simple en utilisant l'espace de nommage **std** comme suit :

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Programmation avec C++ ";
    cout << "Filiere SMP - S 4 ";//affichage sur la même ligne
```

```
    return 0;

}
```

Pour passer à la ligne suivante on utilise **endl** (End of Line : fin de la ligne).

```
cout << "Programmation avec C++ " ;
cout << endl;
cout << "Filiere SMP S 4 " ;
```

On peut écrire le même code en une seule instruction :

```
cout << "Programmation avec C++ " << endl << "Filiere SMP S 4 " ;
```

IV. Description des éléments d'un programme:

IV.1. Variables :

Définition : Un **identificateur** est une suite de caractères parmi :

- Les lettres (minuscules ou majuscules)
- Les chiffres (0, 1, ..., 9)
- Le tiret souligné (_)

dont le premier caractère n'est pas un chiffre.

Exemple :

som1, S_2, max1, sommeFinale.

Définition : Une **variable** sert à mémoriser une valeur. C'est une **zone de mémoire** physique dans la **RAM**. Sa valeur peut changer au cours du temps. Pour manipuler une variable, on lui donne un nom : son identificateur. Lorsqu'on utilise l'identificateur d'une variable, c'est en fait pour manipuler la valeur de la variable.

Une variable est caractérisée par son **type**. Si on déclare une variable de type entier, on ne pourra pas y mettre un réel.

IV.2. Les différents types:

Les nombres et caractères sont stockés dans des variables. Une variable possède un nom et un type adapté. Les principaux types de variable utilisés sont :

- ☐ les entiers, codés sur 8, 16 ou 32 bits :
- ☐ les réels, codés sur 32 ou 64 bits, (double = réel double précision)

- les caractères, codés sur 8 bits, et associés à une lettre via le code ASCII.

IV.2.1. Les entiers :

Le langage C++ distingue plusieurs types d'entiers:

TYPE	DESCRIPTION	TAILLE MEMOIRE
int unsigned int	entier standard signé entier positif	4 octets: $-2^{31} \leq n \leq 2^{31}-1$ 4 octets: $0 \leq n \leq 2^{32}$
short unsigned short	entier court signé entier court non signé	2 octets: $-2^{15} \leq n \leq 2^{15}-1$ 2 octets: $0 \leq n \leq 2^{16}$
char unsigned char	caractère signé caractère non signé	1 octet : $-2^7 \leq n \leq 2^7-1$ 1 octet : $0 \leq n \leq 2^8$

Remarque:

En langage C++, *Un caractère est un entier de 8 bits.*

Exemples:

Les caractères alphanumériques s'écrivent entre ' '. Le caractère '**b**' a pour valeur **98**.

Quelques constantes caractères:

'\n'	Inter ligne
'\t'	tabulation horizontale
'\v'	tabulation verticale
'\r'	retour chariot
'\f'	saut de page
'\\'	backslash
'\"'	cote
'\''	guillemets

IV.2.2. Les réels :

Le langage C++ distingue 2 types de réels:

TYPE	DESCRIPTION	TAILLE MEMOIRE
float	réel standard	4 octets
double	réel double précision	8 octets

IV.3. Déclaration et instruction :

Une **expression** est une suite de composants élémentaires syntaxiquement correcte.

Par exemple :

```
x = 0
(i >= 0) && (i < 10)
```

Une **instruction** est une expression suivie d'un point-virgule.

Un **bloc d'instruction** est formé de plusieurs instructions rassemblées par des accolades { et }.

Par exemple :

```
{
    z = y / x;
    t = y * x;
}
```

Une **déclaration** est une instruction composée d'un **type** et d'une **liste d'identificateurs** séparés par une virgule.

Exemple 1 :

```
char lettre;
int compte;
short entier1, entier2;
float longueur, largeur;
double surface ;
```

Exemple 2 :

```
int a ;
int b = 1, c;
float x, y ;
double z = 2.5;
char c1, c2;
```

En C++, toute variable doit faire l'objet d'une déclaration avant d'être utilisée.

IV.4. Initialisation :

On peut affecter une valeur à une variable au moment de sa déclaration. On dit qu'on initialise la variable.

Exemple :

```
int nbr=2;
char lettre='A'; //'A' désigne le caractère A
```

On peut affecter une valeur à une variable après sa déclaration, on dit qu'on assigne une valeur à une variable.

Exemple :

```
int nbr, triple ;
double longueur ;
nbr=3;
triple = 3 * nbr;
longueur = 0.12; // réel double précision virgule flottante
```

IV.5. Constantes :

Une constante correspond à une valeur littérale. On peut la rencontrer soit sous sa forme littérale (par exemple 250, 3.14, 'A') soit sous la forme d'un identificateur. Chaque fois que l'identificateur de la constante apparaît dans un programme, il est remplacé par **sa valeur qui ne peut pas changer** au cours de l'exécution de celui-ci. On pourra adopter la règle suivante pour les identificateurs de constantes : elles sont composées de majuscules.

Exemple :

```
int main()
{
    const float PI = 3.14159;
    const int TAILLE = 100;
```

```
// ...  
return 0 ;  
}
```

Dans ce cas, le compilateur réserve de la place en mémoire (ici 4 octets), pour la variable PI, on ne peut changer la valeur. On peut associer un modificateur « **const** » à tous les types.

V. Les Opérateurs :

V.1. L'affectation

En C++, l'affectation est un opérateur à part entière. Elle est symbolisée par le signe =. Sa syntaxe est la suivante :

variable = expression ;

Le terme de gauche de l'affectation peut être une variable mais pas une constante. Cette instruction a pour effet d'évaluer *expression* et d'affecter la valeur obtenue à *variable*.

Exemple :

```
#include <iostream>  
using namespace std ;  
int main()  
{  
    int i = 3, j = 2;  
    int r ;  
    r = i + j;  
    cout << r ;  
    return 0;  
}
```

Les instructions sont exécutées séquentiellement, c'est-à-dire les unes après les autres. L'ordre dans lequel elles sont écrites a donc une grande importance.

```
#include <iostream>  
using namespace std ;  
  
int main()  
{  
    int a, b, s ; //déclaration de 3 variables  
    a = 10 ; // affectation  
    b = 50 ; // affectation  
    s = a + b ;  
    s = s + 5 ;  
    cout << " Affichage de a : " << a << "\n";  
    cout << " Affichage de b : " << b << "\n";  
    cout << " Voici le résultat : " << s << "\n";  
}
```



```
    return 0 ;  
}
```

a = 10; signifie « a prend la valeur 10 »

a = 50; signifie « b prend la valeur 50 »

s = a + b; signifie « s prend la valeur a + b »

s = s + 5; signifie « la nouvelle valeur de s est égale à l'ancienne + 5 »

V.2. Les opérateurs arithmétiques

Les opérateurs arithmétiques classiques sont les opérateurs unaires + et - (changement de signe) ainsi que les opérateurs binaires :

+	addition
-	soustraction
*	multiplication
/	division
%	reste de la division (modulo)

Ces opérateurs agissent de la façon attendue sur les entiers comme sur les flottants.

Exercice :

a et b sont des entiers, a = -21430 et b=4782, calculer et afficher a+b, a-b, a*b, a/b, a%b.

V.3. Les opérateurs de comparaison :

>	strictement supérieur
>=	supérieur ou égal
<	strictement inférieur
<=	inférieur ou égal
==	égal
!=	différent

Leur syntaxe est **expression_1 op expression_2**. Les deux expressions sont évaluées puis comparées. La valeur rendue est de type **int** ; elle vaut **1** si la condition est **vraie**, et **0** **sinon**.

Attention à ne pas confondre l'opérateur de test d'égalité **==** avec l'opérateur d'affectation **=**.

Exemple :

```
int a = 10, b = 31 , r1, r2;
r1 = (a == b) ;// r1=0
r2 = (a=b) ; // a=31 et r2= 31
```

V.4. Les opérateurs logiques :

&&	et logique
	ou logique
!	négation logique

Comme pour les opérateurs de comparaison, la valeur retournée par ces opérateurs est un **int** qui vaut 1 si la condition est vraie et 0 sinon.

Exemple :

```
int i=12, j=6, r;
r = ((i >= 0) && (i <= 20) || !(j == 0)); // r=1
...
```

V.5. Les opérateurs d'affectation composée

Les opérateurs d'affectation composée sont :

+= -= *= /= %=

Pour tout opérateur *op*, l'expression ***x op= y*** est équivalente à ***x = x op y***.

Exemple :

```
int a=10,b=2,c=36 ;
b += a ;
c %= b ;
```

V.6. Les opérateurs d'incrément et de décrémentation ++/--

Le langage C++ autorise des écritures simplifiées pour l'incrément et la décrémentation de variables de type entier.

`i = i+1;` est équivalent à `i++`;

`i = i-1;` est équivalent à `i--`;

Les opérateurs d'incrémentation `++` et de décrémentation `--` s'utilisent aussi bien en suffixe (`i++`) qu'en préfixe (`++i`). Dans les deux cas la variable `i` sera incrémentée, toutefois dans la notation suffixe la valeur retournée sera l'ancienne valeur de `i` alors que dans la notation préfixe se sera la nouvelle.

Exemple :

```
int a = 3, b, c;
b = ++a;      /* a et b valent 4 */
c = b++;      /* c vaut 4 et b vaut 5 */
```

V.7. L'opérateur de conversion de type

Attention :

```
double quotient;
quotient = 5/3 ; // 1.00
quotient = 5./3 ; // 1.67
```

Conversion :

L'opérateur de conversion de type, appelé **cast**, permet de modifier explicitement le type d'une variable. On écrit : **(type) objet**

Il peut y avoir perte d'information lors d'une conversion explicite.

Exemples :

```
double reel=2.3;
int partieEntiere;
partieEntiere = (int)reel;
```

```
int i = 3, j = 2;
float x ;
x = (float)i/j; // x=1.5
```

VI. La saisie au clavier : `cin >>`

Pour saisir une donnée au clavier, et la placer dans une variable, on utilise l'instruction **cin>>**, suivie du nom de la variable. La donnée est frappée au clavier suivie de la touche de validation (Entrée)

Le flux de d'entrée standard **cin** (Console Input : entrée console) est définit dans la bibliothèque **iostream**.

Exemples :

```
char alpha;
int i;
float r;
cin >>alpha;    // saisie d'un caractère
cin >>i;         // saisie d'un nombre entier en décimal
cin >>r;         // saisie d'un nombre réel
```

```
int i;
cin>>i;
cout<<"le double de "<<i<<" est : "<<2*i<<endl;
```

On peut également saisir plusieurs valeurs successivement à l'aide d'une seule instruction, lors de la saisie, on sépare les entrées par la touche de validation (Entrée).

```
int cotea, coteb, cotec;
cout<<"Entrez les longueurs des 3 cotes du triangle : "<<endl;
cin>>cotea>>coteb>>cotec;
...
```

Il peut être judicieux, quand on souhaite saisir une donnée, d'en prévenir l'utilisateur par un message :

```
int i;
cout <<"Entrer un nombre entier puis valider" <<endl;
cin >>i;
cout <<"le double de " <<i <<" est : " <<2*i <<endl;
```

VII. Structures alternatives :

Un programme écrit en C++ s'exécute séquentiellement, c'est à dire instruction après instruction.

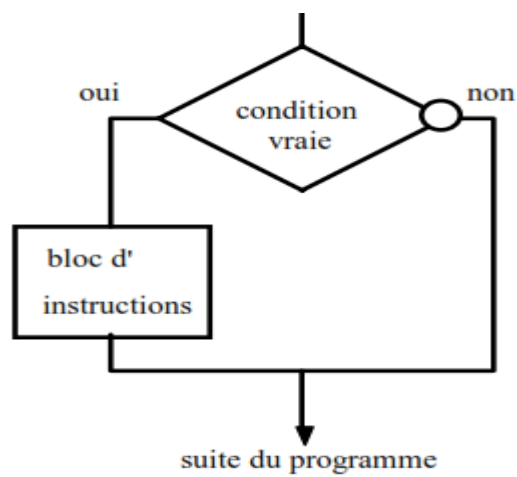
Dans certain programme, où on désire choisir entre deux instructions selon une condition, on utilise la structure alternative **if**.

Syntaxe de **if** :

Forme 1 :

```
if (condition)

    bloc d'instructions
```



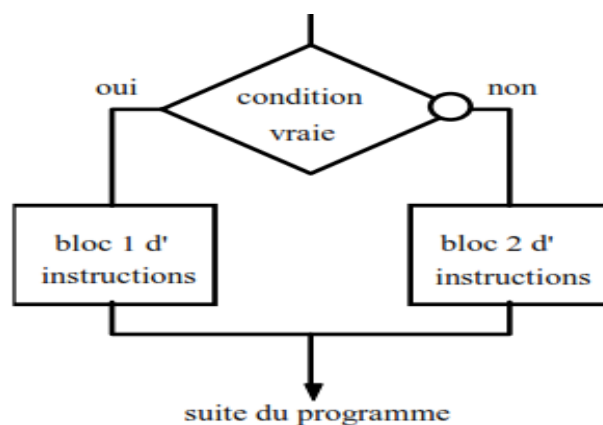
Forme 2 :

```
if (condition)

    bloc 1 d'instructions

else

    bloc 2 d'instructions
```



Exemple1 : if sans else

```
int a =3, b=7, max ;  
  
max=a ;  
  
if (max < b)  
    max = b ;  
  
cout<< max ;
```

Exemple 2 : if avec else

```
int a = -3, valeurAbsolue;  
  
if (a > 0)  
    valeurAbsolue = a ;  
  
else  
    valeurAbsolue = -a;  
  
cout<< valeurAbsolue;
```

Exemple 3 : if imbriquée

```
int a = -3, b= 7, c= 2 , max;  
  
if (a > b)  
    if (a > c)  
        max = a ;  
    else  
        max = c;  
else  
    if ( b > c)  
        max = b ;  
    else  
        max = c;
```

VIII. Structures itératives : Boucles

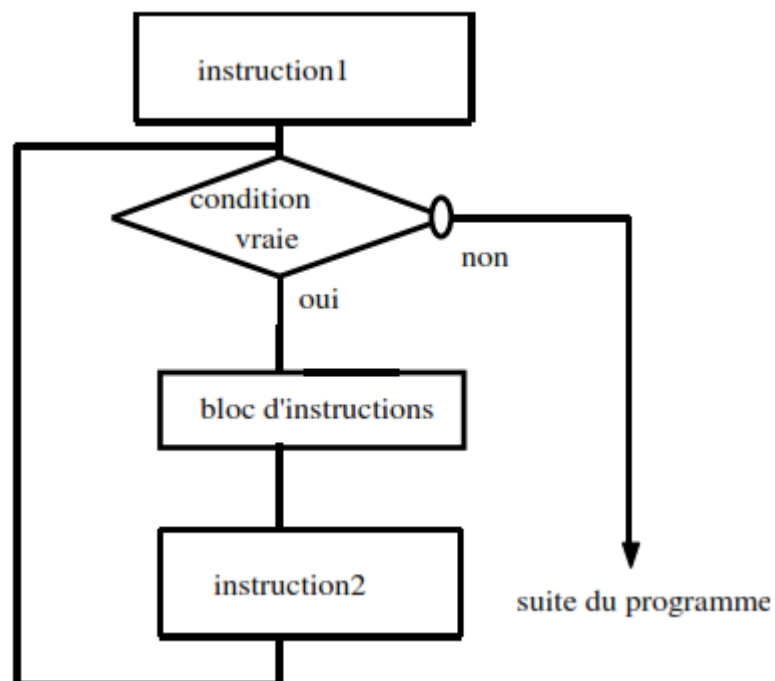
Un programme écrit en C++ s'exécute séquentiellement, c'est à dire instruction après instruction.

Dans certain programme, où on désire répéter un bloc instructions plusieurs fois, on utilise les boucles.

1. Boucle for :

Il s'agit de l'instruction:

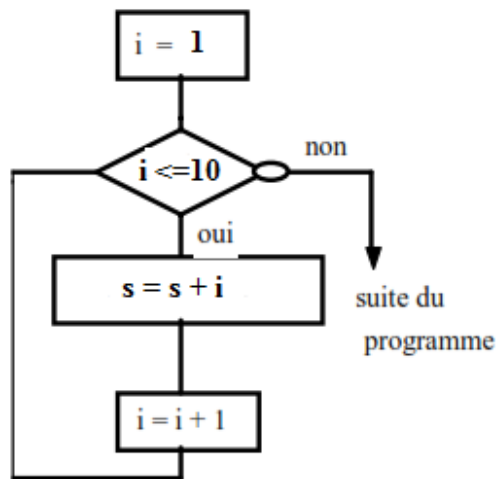
```
for ( instruction1 ; condition ; instruction2)
{
    bloc d'instructions
}
```



Exemple 1 :

$s = 1 + 2 + 3 + \dots + 10$

```
int s = 0 ;
for( int i=1 ; i<=10 ; i++)
    s = s + i;
cout << "la somme est: " <<s;
```



Exemple 2 : Ecrire un programme qui lit un entier n au clavier et affiche ses diviseurs.

```

#include <iostream>
using namespace std ;
int main()
{
    int n ;
    cout <<"Entrez n: ";
    cin >> n;
    cout <<"Les diviseurs de " <<n <<" sont:"<<endl;
    for( int i=1 ; i<=n ; i++)
    {
        if (n%i == 0)
            cout <<i <<endl;
    }
    return 0;
}

```

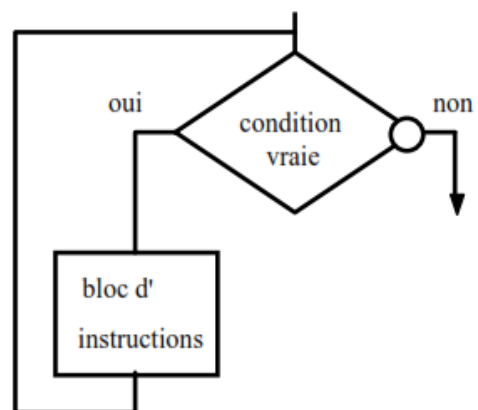
2. Boucle while :

Il s'agit de l'instruction:

```

while ( condition)
{
    bloc d'instructions
}

```



Exemple 1 :

$s = 1 + 2 + 3 + \dots + 10$

```
int s = 0 ;
int i=1 ;
while(i<=10)
{
    s = s + i;
    i++;
}
cout<< "la somme est: " <<s;
```

Exemple 2 : Ecrire un programme qui lit deux entiers a et b au clavier et affiche leur Plus Grand Commun Diviseur (PGCD). On suppose que $a \geq b$.

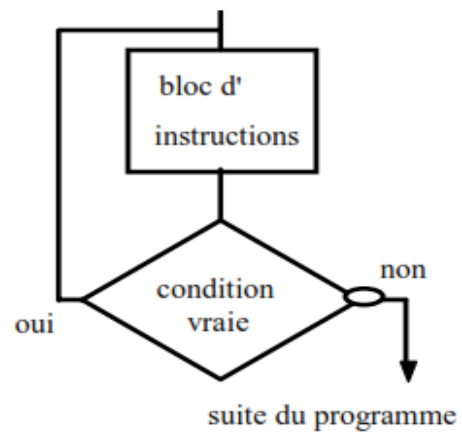
```
#include <iostream>
using namespace std ;
int main()
{
    int a, b , r ;
    cout <<"Entrez a: ";
    cin >> a;
    cout <<"Entrez b: ";
    cin >> b;
    r = a%b;
    while (r!=0)
    {
        a = b;
        b = r;
        r = a%b;
    }
    cout <<"PGCD = " <<b <<endl;
    return 0;
}
```

3. Boucle do ... while :

La syntaxe de la boucle **do ... while** a la forme suivante:

```
do
{
    bloc d'instructions

} while(condition) ;
```



Exemple 1 :

$s = 1 + 2 + 3 + \dots + 10$

```
int s = 0 ;
int i=1 ;
do
{
    s = s + i;
    i++;
} while(i<=10);

Cout << "la somme est: " <<s;
```